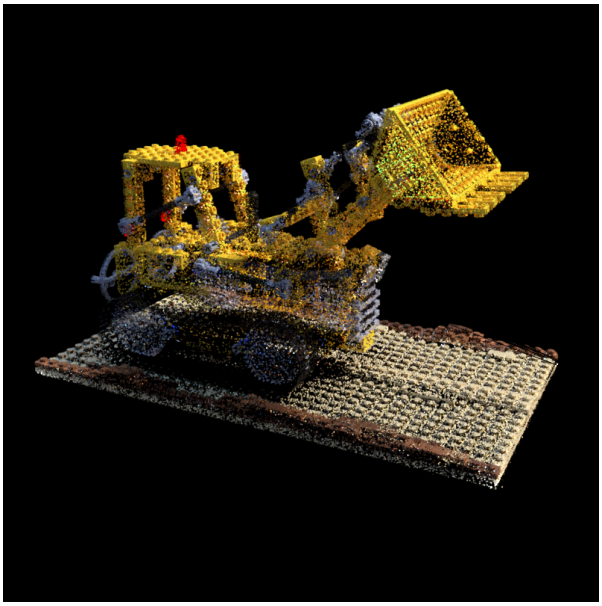
1. Briefly describe your implementation
    a. compute_transmat constructs the transformation matrix T that maps from 2D splat coordinates to pixel coordinates
        i. splat2world is built from two tangent axes of Gaussian (L[0], L[1]) and its world-space center (p_orig) as a 4x3 matrix
        ii. world2ndc is transposed projection matrix
        iii. ndc2pix scales and shifts NDC coordinates to pixel coordinates using focal-length-derived factors W/2 and H/2. These are chained as T = transpose(splat2world) * world2ndc * ndc2pix.
    b. densify_and_split selects Gaussians with high positional gradient AND large scale (>= percent_dense * scene_extent), splits them into N=2 smaller Gaussians sampled around the original, then prunes original
    c. densify_and_clone selects Gaussians with high positional gradient AND small scale (<= percent_dense * scene_extent), and duplicates them in place to better cover under-reconstructed regions
    d. densify_and_prune removes Gaussians that are too transparent (below min_opacity), too large in screen space (above max_screen_size), or too large in world space (above 0.1 * extent)

2. Include some of the representative renderings for different iterations in training. Briefly comment on how the rendering quality is improving during training.
    a. 3000 iterations: Render is already surprisingly clear but has some floaters and incomplete geometric details (such as lego studs)
    b. 12000-21000 iterations: Finer details begin to emerge. Edges sharpen and high-frequency details like the lego studs are more defined. Areas with high volumetric overlap become more defined.
    c. 30000 iterations: The render closely matches the ground truth. Fine details are well-reconstructed and floaters are largely pruned away

3000 iterations	12000 iterations

21000 iterations	30000 iterations

3. Include some images of your scenes with the scale of Gaussians reduced. Are there any patterns you notice with how the learned Gaussians are distributed throughout the scene?
    a. Gaussians concentrate most densely along sharp edges and high-frequency regions such as lego stud boundaries since those regions accumulate high positional gradients and trigger the most cloning/splitting
    b. Areas with less clear texture sparser, larger Gaussians since fewer are needed to represent smooth regions + Gaussians near the surface boundary tend to be smaller and more numerous than those in the interior or background as seen in body of the lego tractor
    c. View-dependent regions have denser Gaussian clusters since the model needs more to capture view-dependent appearances as seen in areas that stick out against the black background (top and shovel of tractor)
    d. Overall, Gaussian density correlates with geometric complexity: more and smaller Gaussians appear where there is fine detail, fewer and larger ones where surfaces are smooth (although this is a bit hard to tell with legos since no surfaces seem super smooth)

	
scaling_modifier = 0.1	scaling_modifier = 0.4
scaling_modifier = 0.7	scaling_modifier = 1.0